

LQCD_Master 核心部分穷尽剖析报告

双代理编排 × 物理任务 × Wick 收缩引擎的代码-物理交叉向深挖

opencode pure

2026 年 8 月 17 日

摘要

本报告对 refer/git-rep/LQCD_Master 做只读穷尽剖析。性质判定：代码-物理交叉向——agent 编排代码（约 26 万行核心）以 LQCD 物理任务（benchmark 15144 行 + skills 物理推导 3348 行）为输入与验证基准，核心机制是”物理任务 → 物理方案 YAML → 物理代码 main.py”的逐符号映射。穷尽枚举 **13 个核心部分候选**（C1-C13），三态分配：深挖 6 / 浅析 4 / 排除 3。最深剖析结论：generate_einsum Wick 收缩引擎（自动 Wick 配对 + γ 代数 + 费米子符号，实测正确输出 $\pi/\rho/\eta_s$ （含 disconnected）/ p/Ω_c 的 einsum）与 双代理 solve/critique/rewrite 循环 + 技能路由注入是两大独特竞争力，支撑 70 任务基准中标准任务近满分（90.0%）的实测表现。

目录

1 项目定位与核心部分清单

1.1 项目性质判定

要点

项目性质：代码-物理交叉向。依据（实测）：**代码体量**编排/工具/提示词/配置代码约 26 万行（run.py + core_architecture 2415 + prompts 527 + utils 260521 + configs 212）；**物理内容** benchmark 物理任务与参考实现 15144 行、skills 物理推导 3348 行；**映射机制**双代理把物理任务转译为物理方案再转译为物理代码，skills 编码 Wick 收缩/谱分解推导，generate_einsum 把物理算符自动映射为 einsum 字符串。代码是载体、物理是输入与标尺，二者靠一一对应的映射链耦合——故判交叉向。

1.2 核心部分总清单（穷尽枚举）

编号	核心部分	处理态	独特性概述
C1	planner+executor 双代理 prompt 工程	深挖	solve/critique/rewrite 循环 + 物理/代码 双层评审
C2	5 个 LQCD 技能设计	深挖	SKILL.md 编码物理推导链 + 代码-物理 映射
C3	benchmark 物理任务与技能调用链	深挖	70 任务自然语言基准 + 手写参考数值
C4	core_architecture 架构设计	深挖	编排器三阶段 + checkpoint 人机协同
C5	run.py 入口与配置体系	深挖	极薄入口 + 配置驱动提交渲染
C6	generate_einsum Wick 收缩引擎	深挖	自动 Wick 配对/ γ 代数/费米子符号的代 码生成器
C7	static_check 静态分析	浅析	AST 级 PyQUDA 语义检查（辅助纠错）
C8	SkillRouter/SkillRunner 路由注入	浅析	技能按阶段注入 LLM（C2 的机制层）
C9	experiments 多骨干实验体系	浅析	GPT-5.4/DeepSeek 对比验证闭环
C10	llm_client / tool_client	浅析	OpenAI 兼容客户端与 web 搜索/执行工 具
C11	10 万行级自动生成 sink 大文件	排除	自动生成产物（C6 输出），非设计部分
C12	docs 既有 pure 报告	排除	第三方分析产物，非库功能
C13	.env / requirements 脚手架	排除	通用样板，无独特竞争力

表 1: 核心部分总清单（三态填满，无”未处理”；数据来源：实
测索引）

2 核心部分深度剖析

六个”深挖”候选按 15 步框架逐部分重现（交叉向：映射表 + 逐符号解释为主，物理元素并入对应步骤）。

2.1 C1 双代理分工与 prompt 工程

- 目的与前期准备.** 双代理把”物理正确性”与”代码正确性”分离验证：Planner 只产物理方案、Executor 只产代码，避免单模型同时承担两难任务（README 定位：Planner 产方案、Executor 产代码，见 README.md:12）。[确证]
- 输入.** Planner 吃自然语言任务 + 固定配置事实 + 技能 (core_architecture/planner.py:72-81); Executor 吃 plan_yaml + summary_md + 输入事实 + 提交配置 (core_architecture/executor.py:102-159)。[确证]
- 输出.** Planner 输出 plan.yaml + summary.md; Executor 输出含 main_program/test_submit/full_submit/notes 四字段的 JSON（输出格式强制 prompts/executor/generate_system.md:140-160）。[确证]
- 整体框架.** 每代理均”生成 → 批判 → 改写”三子阶段：Planner 为 solve/critique/rewrite (core_architecture/planner.py:72-112) , Executor 为 generate/critique/static-check/rewrite

(core_architecture/executor.py:398-524)。[确证]

5. **关键细节.** 批判只标”会导致代码失败或物理错误”的致命问题 (prompts/planner/critique_system.md:6-9); Executor 提示词强制单文件顺序脚本、禁止 main 函数/类/异常处理 (prompts/executor/generate_system.md:34-46); generate_einsum 工具强制调用 + 水印检查 (prompts/executor/generate_system.md:63-82)。[确证]

6. **正确执行.** 每代理生成后经编排器 checkpoint (人工确认/反馈), 未通过则 rewrite_with_feedback 进入下一版本, 上限 max_revision_rounds=3 (core_architecture/orchestrator.py:109-133)。[确证]

7. **实际执行.** 只读侧实测: 69 个核心 py 语法检查 OK=69/FAIL=0; 双代理真实 LLM 调用 **未验证** (需 API key)。[确证/未验证]

8. **实际输出.** 仓库内真实实验目录含 executor_v1 完整产物 (experiments/LQCD_Master_GPT_5.4/2pt_local/task_1/executor_v1/)。[确证]

9. **结果分析.** 70 任务 63 exact / 3 sign-flip / 4 fail (90.0%), 2pt local 20/20 满分; 3pt baryon 仅 73.3% ——重子 3pt 顺序源拓扑是 prompt 工程仍未完全收敛的难点 (experiments/LQCD_Master_GPT_5.4/summary.md:8-30)。[确证]

10. **综合分析.** 对象映射: 自然语言 $\leftrightarrow$ 物理任务; critique $\leftrightarrow$ 同行评审; rewrite $\leftrightarrow$ 论文修改; checkpoint $\leftrightarrow$ 人机协同护栏。深层原因: 把 LQCD 领域协作惯例 (审稿 + 修改 + 确认) 编码为软件流程。[确证/推断]

11. **总结.** 双代理 + 三子阶段 + 人工 checkpoint 构成”自动评审、人机兜底”的工程闭环, 是 90% 级准确率的结构性保证。[确证]

12. **评价.** 优: 物理/代码解耦、可复现; 缺: 批判偏保守 (只拦致命错误)、无符号约定预防机制。[推断]

13. **补充.** prompt 以英文编写、面向 GPT/DeepSeek 类通用 LLM, 未针对推理链/长上下文做显式优化 (推断)。[推断]

14. **参考源.** 见第 5 节 C1 条目。

15. **附录.** prompt 原文片段见第 4 节 S3/S4。

2.2 C2 5 个 LQCD 技能设计

1. **目的与前期准备.** 把领域知识 (物理推导、PyQUDA 用法、分析流程) 编码为可注入 LLM 的独立模块, 替代”硬编码在 prompt 里”的可维护性差的方案。前置: 技能 = 含 YAML front-matter 的 markdown (utils/skill_utils.py:93-111 解析)。[确证]

2. **输入.** 5 个 SKILL.md 共 1926 行: lqcd-analysis(740)/pyquda-tool(614)/ pyquda-gauge(222)/lqcd-physics-correlator(199)/lqcd-physics-spectrum(151) (skills/lqcd-analysis/SKILL.md:1、skills/pyquda-tool/SKILL.md:1)。[确证]

3. **输出.** 注入后的 system prompt 块 (含 Description + body + 内联 reference, 上限 24000 字符, utils/skill_utils.py:215-247)。[确证]

4. **整体框架.** 分工链: correlator (算符/关联函数) $\rightarrow$ spectrum (谱分解/拟合模板) $\rightarrow$ analysis (数据分析); 工具侧 pyquda-tool (传播子/收缩) / pyquda-gauge (纯规范观测量)。物理链尾部显式移交下家 (skills/lqcd-physics-correlator/SKILL.md:36-41)。[确证]

5. 关键细节. correlator 技能给出 Wick 收缩关键公式与 γ_5 -厄米约定 ($\text{prop} = \gamma_5 S^\dagger \gamma_5$) 与 flavor 对称性约化 (skills/lqcd-physics-correlator/SKILL.md:88-98); DeGrand-Rossi 基 γ 矩阵完整列出 (skills/lqcd-physics-correlator/SKILL.md:52-63)。[确证]

代码-物理映射

映射原则: 每个技能中的物理符号都能落到生成的代码符号, 双向可查。

代码符号	物理对象	物理含义与参考源
prop_l/prop_s/prop_c	轻/奇异/粲夸克传播子	$S_l(x, y)$ 等; skills/lqcd-physics-correlator/SKILL.md:23
G5	γ_5	赝标量算符与 γ_5 -厄米约定; skills/lqcd-physics-correlator/SKILL.md:60
epsilon	ϵ^{abc} 颜色反对称张量	重子 diquark 色结构; utils/generate_einsum/hadron_operator.py:20-23
Tmat	投影子	正宇称重子投影 $(1 + \gamma_t)/2$; benchmark/3pt_baryon/task/task_1.txt:3
covDev	协变位移算子	非局域算符的 Wilson 线位移; prompts/executor/generate_system.md:119

表 2: 技能层代码-物理映射 (数据来源: 实测)

6. 正确执行. 技能经 SkillMessageAssembler 拼入 system 消息, reference 目录内容内联 (utils/skill_utils.py:249-262); 路由由 skills.yaml 控制 (configs/skills.yaml:1-10)。[确证]

7. 实际执行. 只读侧逐技能精读 (correlator 全读 + 其余头部); 对 5 个 SKILL.md 统计 front-matter/正文结构 (实测)。真实 LLM 注入效果 **未验证**。[确证/未验证]

8. 实际输出. 每个技能带 reference/ 工作示例 (如 pion/proton mass、wilson loop、baryon 3pt 代码), 是”技能 $\rightarrow$ 代码”的示范性输出 (skills/lqcd-physics-correlator/SKILL.md:132-134)。[确证]

9. 结果分析. 技能路由强制 planner 吃物理技能、executor 吃工具技能 (configs/skills.yaml:4-8), 与实测准确率对照: 纯规范任务 100%、重子 3pt 最低——技能覆盖度与任务难度匹配。[推断]

10. 综合分析. 物理公式 (如 $C_2(t) = \langle O(t)O^\dagger(0) \rangle$, (1)) 以 LaTeX 写入技能, LLM 据此推理; 代码符号映射表 (上文) 保证”物理 $\leftrightarrow$ 代码”逐项可查。[确证]

$$C_2(\vec{p}; t_f, t_i) = \langle \mathcal{O}(\vec{p}, t_f) \mathcal{O}^\dagger(\vec{p}, t_i) \rangle \quad (1)$$

11. 总结. 技能库是”物理知识 $\rightarrow$ 可注入模块”的编译结果, 5 技能构成从观测量到代码的完整推理链。[确证]

12. 评价. 优: 物理推导以标准 LaTeX 呈现、带 reference 示例; 缺: 技能间依赖靠文本移交, 无机制性校验。[推断]

13. 补充. 技能内容本身即”代码-物理交叉”最密集处, 是本库物理承载主体。[确证]

14. 参考源. 见第 5 节 C2 条目。

15. 附录. SKILL.md 关键片段见第 4 节 S6/S7。

2.3 C3 benchmark 物理任务与技能调用链

- 目的与前期准备.** 以”手写标准 PyQUDA 参考实现 + 数值”为 gold standard, 量化 agent 生成代码的正确性 (README.md:118)。前置: C24P29 ensemble、cfg 10000、点源、零动量统一评测条件 (README.md:108)。[确证]
- 输入.** 70 个自然语言任务文本 (2pt_local 20 / 2pt_nonlocal 10 / wilson_loop 12 / 3pt_meson 13 / 3pt_baryon 15, README.md:108-118)。[确证]
- 输出.** standard/benchmark.py (手写参考) + benchmark_data/*.txt (参考数值) 供逐任务比对 (benchmark/2pt_local/standard/benchmark.py:1-30)。[确证]
- 整体框架.** 调用链: task.txt $\rightarrow$ run.py $\rightarrow$ Planner $\rightarrow$ Executor (+skills+generate_einsum) $\rightarrow$ main.py 运行 $\rightarrow$ txt 数据 $\rightarrow$ 与 benchmark_data 对比。[确证]
- 关键细节.** 任务文本既含物理量 ($\bar{u}\gamma_5 d$ 、Cg5、Tmat) 又含数值参数 (stout (1,0.125,4)、tseq=8、点源 [0,0,0,0]), 是”物理-工程”双层指令 (benchmark/3pt_baryon/task/task_1.txt:1-8)。[确证]
- 正确执行.** 每类任务一个 batch 脚本批量跑 (如 experiments/run_scripts/batch_local2pt.sh: 1)。[确证]
- 实际执行.** 只读侧抽样精读 2pt_local/wilson_loop/3pt_meson/3pt_baryon 四类 task_1 + standard 参考头部; 全量 70 任务仅索引。[确证]
- 实际输出.** standard 参考数据含逐任务 t=1 Re 数值 (如 DeepSeek 失败表中 $\eta_s + 1.3067e-01$ 等, experiments/LQCD_Master_DeepSeek/summary.md:20-28)。[确证]
- 结果分析.** Exact 判据 $|\Delta| < 10^{-12}$ 或 $\text{rel} < 10^{-3}$; sign-flip 单独计数 (README.md:131-133)。wilson_loop 全满分说明纯规范任务技能 (pyquda-gauge) 覆盖充分。[确证/推断]
- 综合分析.** 基准既是验证集也是”物理正确性定义”——agent 的”物理”由手写标准编码, 代码-物理对应在此闭环 (benchmark/2pt_local/standard/benchmark.py:1-18)。[确证]
- 总结.** 70 任务基准把”agent 算得对不对”从印象化转为数值可判, 是系统可信度的支柱。[确证]
- 评价.** 优: 任务覆盖 5 类 observable、标准全手写; 缺: 单 ensemble 单组态 (cfg 10000), 统计意义有限 (推断)。[推断]
- 补充.** 任务文本仅 1-10 行, 恰好是”物理-工程双层”最小完备指令集。[确证]
- 参考源.** 见第 5 节 C3 条目。
- 附录.** task_1 原文见第 4 节 S8。

2.4 C4 core_architecture 架构设计

- 目的与前期准备.** 三层 (orchestrator/planner/executor) 封装”总控/方案/代码”三个关注点 (行数 816/379/1219, 职责行数比直观反映”方案最薄、代码最厚”)。[确证]
- 输入.** task + run_dir + llm_client + tool_client + submit_tool + 模式标志 (core_architecture/orchestrator.py:30-49)。[确证]
- 输出.** state dict (planner/executor/test_submission/full_submission 各节) 与 trajectory_full.json (core_architecture/orchestrator.py:62-96)。[确证]
- 整体框架.** run() 三阶段: planner $\rightarrow$ executor $\rightarrow$ test submission (core_architecture/orchestrator.py:74-96)。[确证]

5. **关键细节.** 每阶段 checkpoint 循环 (approved/feedback) 上限 3 轮 (core_architecture/orchestrator.py:109-121); test 提交后 squeue/scontrol 监视 + 动态重写 (core_architecture/orchestrator.py:201-281、core_architecture/orchestrator.py:283-343); executor 缺字段自动反馈重试上限 2 (core_architecture/executor.py:165-197)。[确证]
6. **正确执行.** orchestrator 自身不产生 prompt 内容, 全部委托 planner/executor 与技能/工具层, 只做状态机与 I/O (core_architecture/orchestrator.py:98-107)。[确证]
7. **实际执行.** 只读侧全函数结构 grep + 关键段精读; 运行态执行 **未验证**。[确证/未验证]
8. **实际输出.** 实验产物中 planner/ 与 executor_v1..v2 目录结构即编排器”版本化 + 持久化”设计的落地 (experiments/New_DA_diagonal/planner/plan.yaml:1)。[确证]
9. **结果分析.** executor 静态分析轮数/动态测试轮数由 config 控制 (executor_static_check_rounds=1、executor_dynamic_test_rounds=1, configs/config.yaml:1-2), 架构预留了自动纠错深度的可调性。[推断]
10. **综合分析.** 状态机设计使每一步产物 (plan/bundle/queue/monitor) 皆落盘, 为实验层 5335 文件的可追溯性提供机制保障 (core_architecture/orchestrator.py:815)。[确证/推断]
11. **总结.** 编排器是”薄总控 + 厚实现”, 把可靠性建立在持久化与版本化上。[确证]
12. **评价.** 优: 状态可恢复、监视细节丰富; 缺: orchestrator 816 行承载过多 Slurm 解析逻辑, 可抽独立模块 (推断)。[推断]
13. **补充.** 设计面向单任务串行; 未做任务级并行编排 (推断)。[推断]
14. **参考源.** 见第 5 节 C4 条目。
15. **附录.** orchestrator.run 片段见第 4 节 S1。

2.5 C5 run.py 入口与配置体系

1. **目的与前期准备.** 极薄入口 (133 行) 把”装配 + 启动”与”实现”分离; 配置体系让集群适配不侵入代码。前置: 配置 yaml 需逐集群定制 (README.md:150-157)。[确证]
2. **输入.** CLI 五参数 (-test/-run-dir/-task/-list-skills/-non-interactive, run.py:17-25) + .env (API key) + configs 三 yaml。[确证]
3. **输出.** run_dir (默认 runs/< 时间戳 >) 下的全部产物 (run.py:28-32)。[确证]
4. **整体框架.** main() 装配四组件后调 orchestrator.run(), 结束打印轨迹路径与版本/提交摘要 (run.py:90-129)。[确证]
5. **关键细节.** -task 支持文件 (.txt 后缀自动读入, run.py:95-98); 交互输入用 prompt_toolkit、回退 input (run.py:61-71); config.yaml 的 slurm/runtime/ensemble 节直接驱动提交脚本渲染 (core_architecture/executor.py:1113-1160)。[确证]
6. **正确执行.** 快速冒烟: python run.py -list-skills (只列技能, 不发 LLM 请求); 全流程: python run.py -task ”< 文本 >” -test -non-interactive (README.md:26-28)。[确证]
7. **实际执行.** 只读侧实测: CLI 结构精读、config 三文件全文/结构精读; 实际运行 (需 API+ 集群) **未验证**。[确证/未验证]
8. **实际输出.** run.py 打印的 JSON 摘要字段 (planner/executor 版本、test/full submission) 与实际实验目录结构一致 (run.py:124-129)。[确证]